

NYP Events Application



Final Report

5 March 2019 – 24 May 2019

Goh Pin Kai 171067G

Version 1.2

CONTENTS	PAGE
PROJECT INTRODUCTION	3 to 4
BACKGROUND	3 to 4
PROJECT OBJECTIVE	5
PROJECT SCOPE	5
TARGET AUDIENCE	5
KEY FEATURES	5
EXCLUSION	5
MAJOR DELIVERABLES	6
PROJECT DEVELOPMENT	7
PROJECT DEVELOPMENT TIMELINE	7
PROJECT SPECIFICATION	7
SOFTWARE/HARDWARE/TECHNOLOGIES USED	6 to 8
ROLES AND RESPONSIBILITIES	13
ASSUMPTIONS AND CONSTRAINTS	13
PROJECT SCHEDULE	13
PROJECT REVIEW	13 to 14

Project Introduction

Background Information

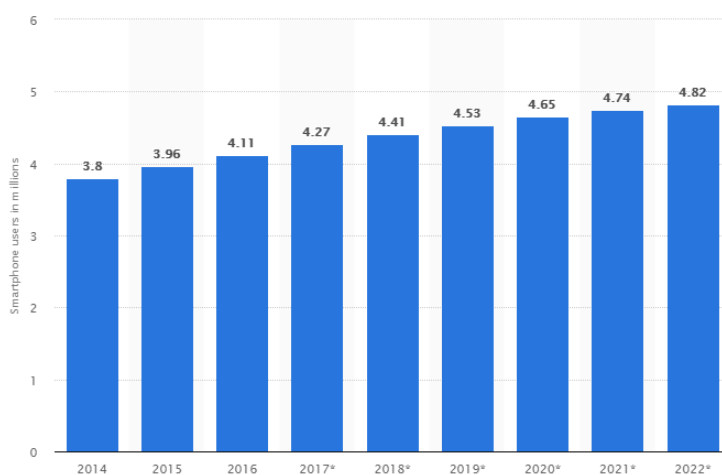
Event posters use paper, which is not environmentally friendly.

Event posters may not appeal to every student because they may be placed in secluded areas.

When the event is over, the event poster has to be taken down by hand; otherwise, it will be left there forgotten until someone takes it down.

Additionally, a smartphone is widely used around the world and towards many people; it may have become a necessity in today's day and age. With a smartphone, you will be able to download mobile applications with functions suited to cater to your needs. Hence, a mobile application is the core of a smartphone that sets it apart from a mobile phone where its only purpose is to make phone calls and send text messages.

Number of smartphone users in Singapore from 2014 to 2022 (in millions)*



© Statista 2019

[About this statistic](#)

[Show source](#)

[DOWNLOAD](#)[SETTINGS](#)[SHARE](#)

PNG

PDF

XLS

PPT

[DESCRIPTION](#)[SOURCE](#)[MORE INFORMATION](#)

This statistic shows the number of smartphone users in Singapore from 2014 to 2022. For 2017, the number of smartphone users in Singapore is estimated to reach 4.27 million.

Source: <https://www.statista.com/statistics/494598/smartphone-users-in-singapore/>

What makes a mobile application so important? Firstly, a mobile application allows freedom in designing. A mobile app can be designed with a lot of elaborate functions, based on advanced gestures like 'tap', 'swipe', 'drag', 'pinch', 'hold' and more.

Moreover, these apps can use these gestures to offer innovative functionality that can help users perform a task better, enhancing user experience. For an instance, an app can let users move to a next or previous step using the swipe gesture.

Furthermore, Apps can work faster than websites. A well-designed mobile app can perform actions much quicker than a mobile website. Apps usually store their data locally on mobile devices, in contrast to websites that generally use web servers. For this reason, data retrieval happens swiftly in mobile apps. Additionally, apps can further save users' time by storing their preferences, and using them to take proactive actions on users' behalf. Therefore, websites use JavaScript code to perform most of their functions. The framework that mobile apps use can run almost five times faster than a JavaScript code.

Project Objective

To develop a mobile application for students and lecturers to view events, host an event and register for the event.

Project Scope

Target Audience

With this project, we are targeting two main groups of users. The first group would be the students of NYP (Nanyang Polytechnic), and the second would be the lecturers of NYP.

The students and lecturers can host events in NYP. The lecturers can facilitate the events by giving students approval to host events. The students would be the main users of mobile application.

Key Functions

An interactive mobile application in NYP would be a package that encompasses various features of an events management application.

Key Features

1. View the events happening in NYP
2. Register for events in NYP

3. Host events in NYP

Major Deliverables

- Project Poster
- NYP Events Manager Application UI/UX Design
- Database Design
- Database
- Laravel RESTful API
- NYP Events Manager Application
- Application Documentation
- Week 12 Presentation
- Weekly Progress Report
- Final Report

Project Development

Stage	Work
Phase 1 (Week 1 – Week 6)	User Interface (UI) Design
Phase 2 (Week 7 – Week 8)	Back-End Development: REST API using PHP Framework (Laravel)
Phase 3 (Week 9 – Week11)	Front-End Development: React JS

Project Development Timeline

Phase 1 -> 5 March 2019 to 12 April 2019

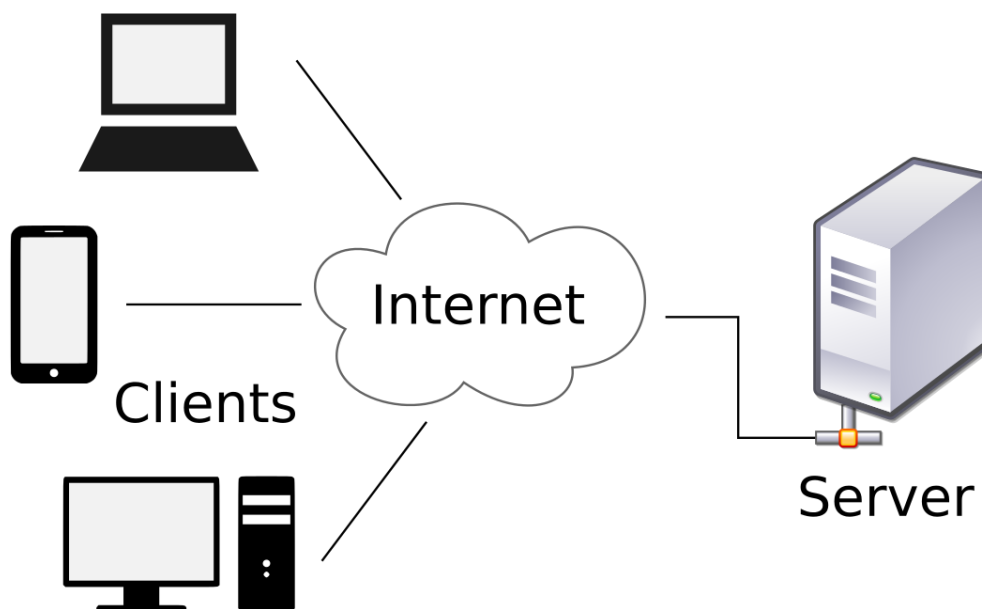
Phase 2 -> 15 April 2019 to 26 April 2019

Phase 3 -> 29 April 2019 to 17 May 2019

The timeline of our project is 3 months. The date of commencement is 5th March 2019. The project is expected to be finished before 21st May. Therefore, the time is of essence.

System Architecture

System Architecture Diagram:



Server: XAMPP Server with Apache with MySQL

API: Laravel API is exposed to the internet

Client: The API can be used by the clients with a bearer token

Project Specifications

Software/Hardware/Technologies Used

Visual Studio Code

This will be used to code the application programming interface (API) of the application and the web application using ReactJS.

User Interface Design

UI/UX Design

1. Adobe XD

To design a UI, this would enable the visualisation of the mobile application and the possible implementation of animation, pictures and colours before development of the application.

Back-End Development

Database

1. Visual Paradigm

This is used to create the database design with purpose of showing how the database layout will look like. The database design is designed to easily show us the database name and attributes so as to create the database with ease where it details the requirements of the database.

2. XAMPP

This is a free and open-source cross-platform web server solution stack package for scripts written in the PHP programming language which will be used. It will be the server used where the mobile application will store the data of the users and records.

API Development

3. Laravel

It is a PHP Web Framework that is used to develop the web application and it will be used to access the relational databases. An API simplifies programming by abstracting the underlying

implementation and only exposing objects or actions the developer needs.

4. Composer

This is an application-level package manager for the PHP programming language that provides a standard format for managing dependencies of PHP software where Laravel will be used and required libraries.

5. Postman

This is a type of software testing involving testing the API directly. It enables a testing of the API to see if it meets the expectations for functionality, reliability, performance and security of a RESTFUL API.

Front-End Development

Web Application Development

1. React.js

This will be used to create the mobile application that will be interacting the API created. This application will be using JavaScript as the programming language.

2. Node.js

Node.js uses an event-driven, non-blocking I/O model that makes it lightweight and efficient. Express is a framework based on NodeJS that is used to create server side for an app.

3. Yarn

Yarn has two main advantages over NPM. Firstly, Yarn creates a yarn.lock file. This file stores the exact versions of dependencies to the last digit. When installing, Yarn first checks the lock file for the versions, then checks package.json file.

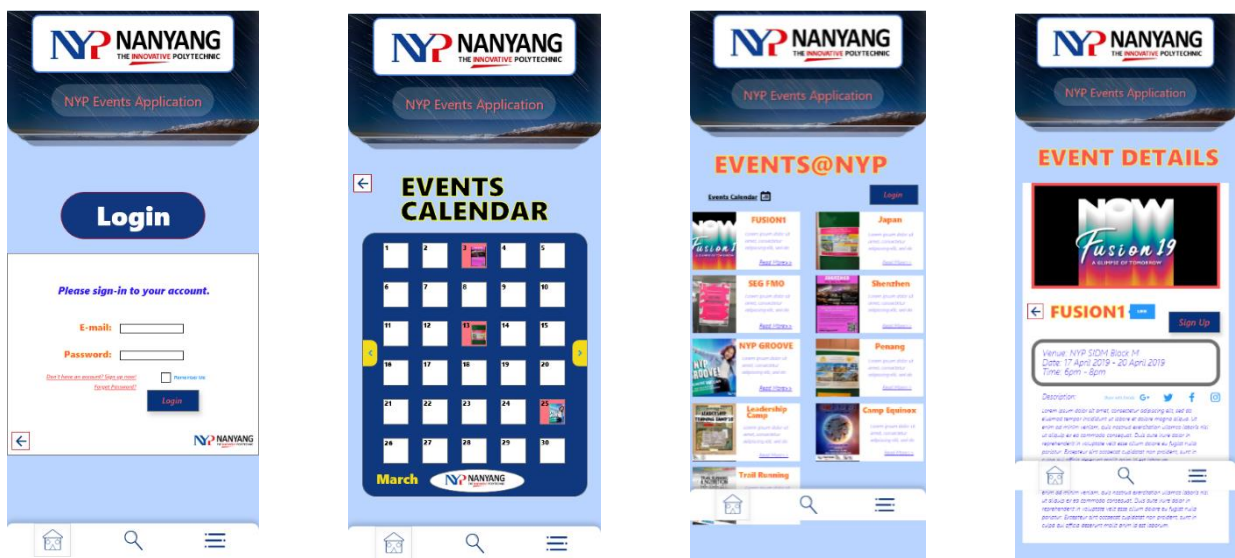
1st Phase: UI Design

UI Design



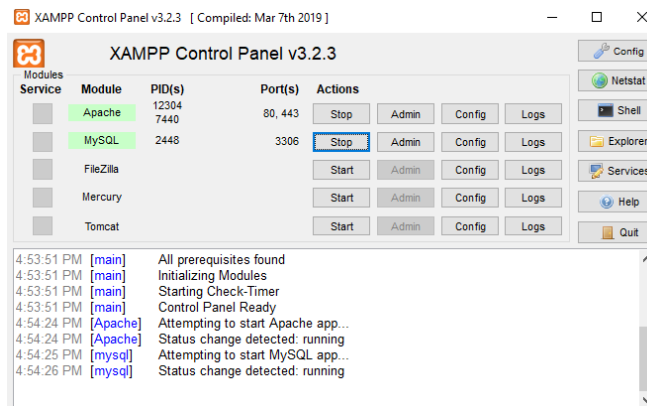
Scan Me!

- ❖ The user interface of the mobile with animation using Adobe XD.

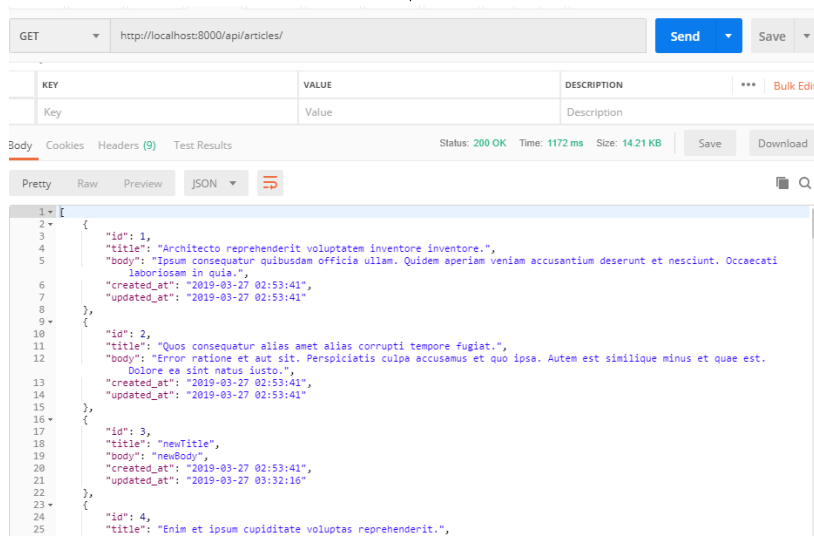


2nd Phase: Development of REST API

- ❖ Created a functioning Representational state transfer (RESTful) Application Programming Interface (API)



```
C:\Users\171067G>cd\users\171067g\nypapp2
C:\Users\171067G\nypapp2>php artisan serve
Laravel development server started: <http://127.0.0.1:8000>
```



RESTful API

- RESTful API stands for 'Representational state transfer'
- A RESTful API is an application program interface (API) that uses HTTP requests to GET, PUT, POST and DELETE data.

Benefits of a RESTful API

 *The client-server constraint works on the concept that the client and the server should be separate from each other*

- This allows the client and the server to evolve individually and independently. For an example, from my experience, creating a PHP webpage requires the front-end and back-end to work together but with this restful API, the back-end and front-end side can be developed separately and they do not depend on each other.

 *RESTful APIs are stateless*

- Being **stateless** makes **REST API** less complex – by removing all server-side state synchronization logic. A **stateless API** is also easy to cache as well. The server never loses track of “where” each client is in the application because the client sends all necessary information with each request.

API Token

```
2019_04_23_021501_adds_api_token_to_users_table.php
1 <?php
2
3 use Illuminate\Support\Facades\Schema;
4 use Illuminate\Database\Schema\Blueprint;
5 use Illuminate\Database\Migrations\Migration;
6
7 class AddsApiTokenToUsersTable extends Migration
8 {
9     /**
10      * Run the migrations.
11      *
12      * @return void
13      */
14     public function up()
15     {
16         Schema::table('users', function (Blueprint $table) {
17             $table->string('api_token', 60)->unique()->nullable();
18         });
19     }
20
21     public function down()
22     {
23         Schema::table('users', function (Blueprint $table) {
24             $table->dropColumn('api_token');
25         });
26     }
27 }
28
```

This creates a bearer token for a user which only allows users that has a bearer token to access the API to retrieve data.

Migrations of tables to database

Category.php
Event_Feedback.php
Event_Register.php
Event.php
User.php

migrations
2019_04_22_074803_create_users_table.php
2019_04_22_074815_create_categories_table.php
2019_04_22_074818_create_events_table.php
2019_04_22_074848_create_event_registers_table.php
2019_04_22_074912_create_event_feedback_table.php
2019_04_23_021501_adds_api_token_to_users_table.php

```
>php artisan migrate
```

Table	Action	Rows	Type	Collation	Size	Overhead
categories	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	16 K1B	-
events	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	48 K1B	-
event_feedback	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48 K1B	-
event_registers	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48 K1B	-
migrations	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_unicode_ci	16 K1B	-
users	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	48 K1B	-
6 tables	Sum	15	InnoDB	latin1_swedish_ci	224 K1B	0 B

Examples of a class with foreign key reference created

```
1 <?php
2
3 namespace App;
4
5 use Illuminate\Database\Eloquent\Model;
6
7 class Event extends Model
8 {
9
10     protected $primaryKey = 'event_id';
11     protected $fillable = [
12         'event_title', 'event_desc', 'startdate', 'enddate', 'starttime',
13         'endtime', 'organizer', 'venue', 'u_id', 'c_id', 'photo'
14         , 'organizer_email', 'status' => 'boolean: false'
15     ];
16     public function users()
17     {
18         return $this->belongsToMany('Models\User', 'user_id', 'id');
19     }
20     public function categories()
21     {
22         return $this->belongsToMany('Models\Category', 'c_id', 'category_id');
23     }
24     public function event_registers()
25     {
26         return $this->hasMany('Models\Event_Register', 'e_id', 'event_id');
27     }
28     public function event_feedback()
29     {
30         return $this->hasMany('Models\Event_Feedback', 'e_id', 'event_id');
31     }
32 }
```

```
1 <?php
2
3 namespace App;
4
5 use Illuminate\Database\Eloquent\Model;
6 use Illuminate\Foundation\Auth\User as Authenticatable;
7
8 class User extends Authenticatable
9 {
10     protected $table = 'users';
11     protected $primaryKey = 'user_id';
12
13     /**
14      * The attributes that are mass assignable.
15      *
16      * @var array
17      */
18     protected $fillable = [
19         'name', 'email', 'password'
20     ];
21
22     public function events()
23     {
24         return $this->hasMany('Models\Event', 'u_id', 'user_id');
25     }
26     public function event_registers()
27     {
28         return $this->hasMany('Models\Event_Register', 'u_id', 'user_id');
29     }
30     public function event_feedback()
31     {
32         return $this->hasMany('Models\Event_Feedback', 'u_id', 'user_id');
33     }
34
35     /**
36      * The attributes that should be hidden for serialization.
37      *
38      * @var array
39      */
40     protected $hidden = [
41         'password', 'remember_token'
42     ];
43
44     public function generateToken()
45     {
46         $this->api_token = str_random(40);
47         $this->save();
48
49         return $this->api_token;
50     }
51 }
```

```
1 <?php
2
3 namespace App;
4
5 use Illuminate\Database\Eloquent\Model;
6
7 class Category extends Model
8 {
9     protected $table = 'categories';
10     protected $primaryKey = 'category_id';
11
12     protected $fillable = [
13         'category_desc'
14     ];
15
16     public function events()
17     {
18         return $this->hasMany('Models\Event', 'c_id', 'category_id');
19     }
20
21 }
22
```

Adding Foreign Key in Database

```
1 <?php
2
3 use Illuminate\Support\Facades\Schema;
4 use Illuminate\Database\Schema\Blueprint;
5 use Illuminate\Database\Migrations\Migration;
6
7 class CreateEventTable extends Migration
8 {
9     /**
10      * Run the migrations.
11      *
12      * @return void
13      */
14     public function up()
15     {
16         Schema::create('events', function (Blueprint $table) {
17             $table->increments('event_id');
18             $table->string('event_title');
19             $table->string('event_desc');
20             $table->date('startdate');
21             $table->date('enddate');
22             $table->string('starttime');
23             $table->string('endtime');
24             $table->string('organizer');
25             $table->string('venue');
26             $table->integer('u_id')->unsigned();
27             $table->foreign('u_id')->references('user_id')
28                 ->on('users')->onDelete('cascade');
29             $table->integer('c_id')->unsigned();
30             $table->foreign('c_id')->references('category_id')
31                 ->on('categories')->onDelete('cascade');
32             $table->string('organizer_email');
33             $table->string('status');
34             $table->date('closing_date');
35             $table->date('cancellation_date')->nullable();
36             $table->timestamps();
37         });
38     }
39
40     /**
41      * Reverse the migrations.
42      *
43      * @return void
44      */
45     public function down()
46     {
47         Schema::dropIfExists('events');
48     }
49 }
```

```
1 2019_04_22_074646_create_event_registers_table.php
2
3 <?php
4
5 use Illuminate\Support\Facades\Schema;
6 use Illuminate\Database\Schema\Blueprint;
7 use Illuminate\Database\Migrations\Migration;
8
9 class CreateEventRegistersTable extends Migration
10 {
11     /**
12      * Run the migrations.
13      *
14      * @return void
15      */
16     public function up()
17     {
18         Schema::create('event_registers', function (Blueprint $table) {
19             $table->increments('id');
20             $table->foreign('e_id')->references('event_id')
21                 ->on('events')->onDelete('cascade');
22             $table->integer('u_id')->unsigned();
23             $table->foreign('u_id')->references('user_id')
24                 ->on('users')->onDelete('cascade');
25             $table->integer('c_id')->unsigned();
26             $table->foreign('c_id')->references('category_id')
27                 ->on('categories')->onDelete('cascade');
28             $table->string('register_date');
29             $table->date('cancellation_date')->nullable();
30             $table->timestamps();
31         });
32     }
33
34     /**
35      * Reverse the migrations.
36      *
37      * @return void
38      */
39     public function down()
40     {
41         Schema::dropIfExists('event_registers');
42     }
43 }
```

```
1 2019_04_22_074912_create_event_feedback_table.php
2
3 <?php
4
5 use Illuminate\Support\Facades\Schema;
6 use Illuminate\Database\Schema\Blueprint;
7 use Illuminate\Database\Migrations\Migration;
8
9 class CreateEventFeedbackTable extends Migration
10 {
11     /**
12      * Run the migrations.
13      *
14      * @return void
15      */
16     public function up()
17     {
18         Schema::create('event_feedback', function (Blueprint $table) {
19             $table->increments('id');
20             $table->foreign('e_id')->references('event_id')
21                 ->on('events')->onDelete('cascade');
22             $table->integer('u_id')->unsigned();
23             $table->foreign('u_id')->references('user_id')
24                 ->on('users')->onDelete('cascade');
25             $table->integer('c_id')->unsigned();
26             $table->foreign('c_id')->references('category_id')
27                 ->on('categories')->onDelete('cascade');
28             $table->string('feedback_desc');
29             $table->integer('rating');
30             $table->timestamps();
31         });
32     }
33
34     /**
35      * Reverse the migrations.
36      *
37      * @return void
38      */
39     public function down()
40     {
41         Schema::dropIfExists('event_feedback');
42     }
43 }
```

Purpose of an API

- + *An Application Program Interface (API) is a set of routines, protocols, and tools for building software application.*
- + *Basically, an API specifies how software components should interact.*

3rd Phase: Front-End Development using React JS to consume the API

React.JS

- + ***React.JS** basically is an open-source JavaScript library.*
- + ***React.JS** is a single-page application (SPA).*
- + ***React.JS** allows the creation of reusable user interface components.*
- + *The main purpose of **React.JS** is to be fast, scalable, and simple.*

Virtual DOM

- + *DOM stands for “Document Object Model”.*
- + *Virtual DOM is in-memory representation of Real DOM.*
- + *Enhancement of Performance*

Virtual DOM Technicality

- + *Updating the browser’s DOM is a three-step process in React.JS.*
 1. Whenever anything may have changed, the entire UI will be re-rendered in a Virtual DOM representation.
 2. The difference between the previous Virtual DOM representation and the new one will be calculated.
 3. The real DOM will be updated with what has actually changed. This is very much like applying a patch.

Data retrieved from the database through an API by doing a Fetch

```
15   componentDidMount() {
16     this.setState({loading: true})
17     fetch('http://nypapp.eventsmgr/api/events')
18       .then(data => data.json())
19       .then(data => this.setState({data, loading: false}))
20   }
21
Route::get('users', 'UserController@index');
Route::get('users/{user_id}', 'UserController@show');
Route::post('users', 'UserController@store');
Route::put('users/{user_id}', 'UserController@update');
Route::delete('users/{user_id}', 'UserController@delete');

Route::get('events', 'EventController@index');
Route::get('events/{event_id}', 'EventController@show');
Route::post('events', 'EventController@store');
Route::put('events/{event_id}', 'EventController@update');
Route::delete('events/{event_id}', 'EventController@delete');

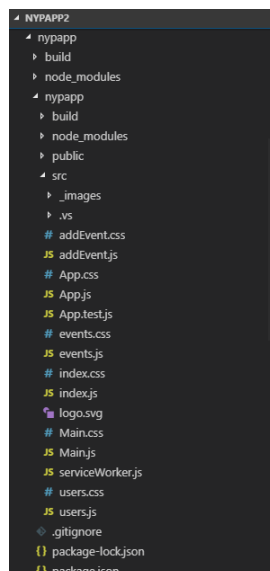
Route::get('categories', 'CategoryController@index');
Route::get('categories/{category_id}', 'CategoryController@show');
Route::post('categories', 'CategoryController@store');
Route::put('categories/{category_id}', 'CategoryController@update');
Route::delete('categories/{category_id}', 'CategoryController@delete');

Route::get('event_feedback', 'FeedbackController@index');
Route::get('event_feedback/{id}', 'FeedbackController@show');
Route::post('event_feedback', 'FeedbackController@store');
Route::put('event_feedback/{id}', 'FeedbackController@update');
Route::delete('event_feedback/{id}', 'FeedbackController@delete');

Route::get('event_registers', 'EventRegisterController@index');
Route::get('event_registers/{id}', 'EventRegisterController@show');
Route::post('event_registers', 'EventRegisterController@store');
Route::put('event_registers/{id}', 'EventRegisterController@update');
Route::delete('event_registers/{id}', 'EventRegisterController@delete');
Route::middleware('auth:api')
->get('users', function (Request $request) {
    return $request->user();
});
Auth::guard('api')->user(); // Instance of the logged user
Auth::guard('api')->check(); // if a user is authenticated
Auth::guard('api')->id(); // the id of the authenticated user

Route::post('register', 'Auth\RegisterController@register');
Route::post('login', 'Auth\LoginController@login');
Route::post('logout', 'Auth\LoginController@logout');
```

Single-Page Application



```
1 import React, { Component } from "react";
2 import {
3   Route,
4   NavLink,
5   HashRouter
6 } from "react-router-dom";
7 import users from "../users";
8 import events from "../events";
9 import addEvent from "../addEvent";
10 import editEvent from "../editEvent";
11 import "../Main.css";
12
13 class Main extends Component {
14
15   render() {
16     return (
17       <HashRouter>
18         <div>
19           <header></header>
20           <ul className="nav">
21             <li className="nav"><NavLink to="/">Home</NavLink></li>
22             <li className="nav"><NavLink to="/events">Events</NavLink></li>
23             <li className="nav"><NavLink to="/users">Users</NavLink></li>
24           </ul>
25           <div className="content">
26             <Route path="/users" component={users}/>
27             <Route path="/events" component={events}/>
28             <Route path="/addEvent" component={addEvent}/>
29             <Route path="/editEvent" component={editEvent}/>
30           </div>
31         </div>
32       </HashRouter>
33     );
34   }
35 }
36
37 export default Main;
```

Project Schedule

The schedule can be found in Annex A of this report.

Project Poster

The schedule can be found in Annex B of this report.

Project Review

The progress of our project will be reviewed by our project supervisor as well as the managers.

There would be three assessments. First, there is an introduction on week 2.

Next, there will be an Interim Assessment on week 8.

Finally, last but not least, a Final Assessment will be held on week 12.

A project plan would be submitted during the interim assessment, to update our project supervisor as well as the managers about our progress thus far. This gives them a quick run-down on what the project is about.

A project report that comprises of the project plan as well as the documentation/manual will to be submitted at the end of the project

Conclusion



Obstacles that I have faced

- Time-Constraint
 - Errors
 - Stress
 - Distractions
- ✓ This FYP has taught me a lot of things and allowed me to gain new experiences but it has also posed several obstacles that I have to overcome for me to evolve. The few problems that I have face were errors where it has caused many issues with my application that frustrates me a lot. Moreover, the time-constraint also posed as a challenge due to unforeseen circumstances. Even though, these factors have induced stress which mentally exhausted me but I pressed on with my sheer willpower and determination to getting good grades for this FYP. Distractions have also gradually slowed down my progress but I try my best to focus and accomplish my tasks each week. Tough: Completing my API and application under 6 weeks



What Have I learnt from this Final Year Project?

- Designing a User Interface with Adobe XD
 - RESTful API
 - React.JS Application
 - Discipline
 - Single-Mindedness
 - Perseverance
- ✓ I have learnt to design an application UI with a new software, how to create an RESTful API and create an application that can consume the API and fetch data from the database. This FYP has also taught me discipline by forcing myself to wake up early and reach school before 8:30 am. I have also learnt how to be single-minded and persevere through the tedious FYP despite all the distractions and obstacles I faced. During this FYP, I have applied my research skill substantially into solving my problems such as errors and learning a new skill set with the use of Google, YouTube and Lynda.com. This has proven to me that the never giving-up attitude plays a crucial role in this FYP.

Annex A – Project Schedule



Annex B – Project Poster

NYP Events Manager Application



Objectives:

To develop an application for students and lecturers of Nanyang Polytechnic(NYP) to view and manage events in NYP.

Description:

A mobile application developed to inform students of NYP about the events happening in NYP. This application allows students and lecturers all with the convenience of a mobile phone to:

- View events
- Host events
- Register for events

Benefits:

1. Modern
2. Easily Accessible
3. Update Events Instantly
4. Interactive
5. Fast & Efficient



Scan Me!

Student:
171067G Goh Pin Kai

Supervisor:
Ms Kang Siew Hua

NYP NANYANG
THE INNOVATIVE POLYTECHNIC

SCHOOL OF ENGINEERING
think innovation, create possibilities

End of Report